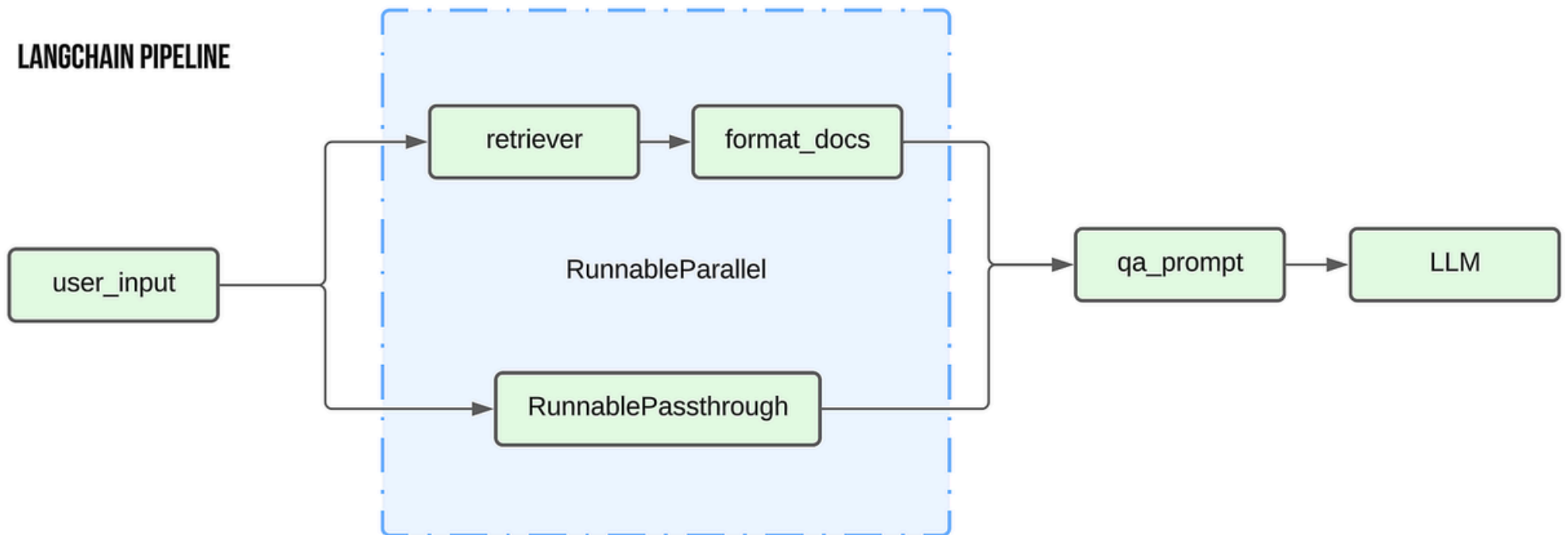


Complete Notes on LangChain Expression Language(LCEL)

LANGCHAIN PIPELINE



Use-cases Template apps

[Chat-Langchain](#)

[Web-Langchain](#)

[More...](#)



Chains Expression language

[RAG Chain](#)

[Chat](#)

[SQL Chain](#)

[More...](#)



Building Blocks Open source

Document
Loaders

Document
Transformers

Vectorstores

Embedding
Models

Prompts

LLMs



Platform Observability, Data Management, Eval

[LangSmith](#) + [Prompt Hub](#)



1. What is LCEL

LangChain Expression Language (LCEL) is a declarative way to compose LLM applications using LangChain.

It allows developers to build **chains, pipelines, and agent workflows** by combining components with simple operators.

LCEL focuses on:

- composability
- streaming support
- async execution
- observability
- parallelism
- production-ready pipelines

Key idea:

Build AI workflows by **connecting components into runnable pipelines**.

2. Core Concept: Runnable

Everything in LCEL is a **Runnable**.

A Runnable is an object that can:

- take input
- process it
- return output

Common runnable components:

Component	Description
PromptTemplate	Formats prompts
LLM / ChatModel	Generates responses
Retriever	Fetches documents
OutputParser	Structures output
Lambda	Custom logic

Example pipeline:

```
</> Python
prompt | llm | parser
```

Each stage passes output to the next stage.

3. LCEL Pipe Operator

LCEL uses the **pipe operator**.

Python

|

It chains components sequentially.

Example:

Python

```
chain = prompt | llm | StrOutputParser()
```

Flow:

User Input → Prompt → LLM → Parser → Output

4. Basic LCEL Chain Example

Example summarization chain:

Python

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI()

prompt = ChatPromptTemplate.from_template(
    "Summarize the following text:\n{text}"
)

chain = prompt | model | StrOutputParser()

chain.invoke({"text": "LangChain is a framework for building LLM apps."})
```

4. Basic LCEL Chain Example

Example summarization chain:

```
</> Python

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatOpenAI()

prompt = ChatPromptTemplate.from_template(
    "Summarize the following text:\n{text}"
)

chain = prompt | model | StrOutputParser()

chain.invoke({"text": "LangChain is a framework for building LLM apps."})
```

5. Runnable Interface

Every LCEL component supports these methods.

Method	Purpose
invoke()	Single execution
batch()	Batch processing
stream()	Token streaming
ainvoke()	Async execution
abatch()	Async batch

Example:

```
</> Python

chain.invoke(input)
```

Streaming:

```
</> Python
```

```
for chunk in chain.stream(input):  
    print(chunk)
```

Input → multiple chains → outputs combined.

Output example:

```
</> Python
```

```
{  
    "summary": "...",  
    "keywords": [...]  
}
```

6. RunnableLambda

Allows custom Python logic inside chains.

Example:

```
</> Python
```

```
from langchain_core.runnables import RunnableLambda  
  
def uppercase(x):  
    return x.upper()  
  
chain = RunnableLambda(uppercase)
```

Pipeline example:

```
</> Python
```

```
prompt | llm | RunnableLambda(process_response)
```

8. RunnablePassthrough

Passes input forward unchanged.

Useful when combining inputs.

Example:

```
</> Python
```

```
from langchain_core.runnables import RunnablePassthrough  
  
chain = RunnablePassthrough()
```

Common use:

```
</> Python
```

```
{  
    "context": retriever,  
    "question": RunnablePassthrough()  
}
```

7. RunnableParallel

Runs components in parallel.

Example:

```
</> Python
```

```
from langchain_core.runnables import RunnableParallel  
  
chain = RunnableParallel(  
    summary=summarizer_chain,  
    keywords=keyword_chain  
)
```

9. RunnableMap

Transforms inputs into structured outputs.

Example:

```
</> Python
```

```
chain = {  
    "summary": summary_chain,  
    "sentiment": sentiment_chain  
}
```

Equivalent to parallel execution.

10. Prompt Templates in LCEL

Prompt templates structure inputs.

Example:

```
</> Python

from langchain_core.prompts import PromptTemplate

prompt = PromptTemplate.from_template(
    "Explain the concept of {topic} in simple terms."
)
```

Use in chain:

```
</> Python

chain = prompt | llm
```

Invocation:

```
</> Python

chain.invoke({"topic": "transformers"})
```

11. Chat Prompt Templates

Used with chat models.

Example:

```
</> Python

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an AI tutor."),
    ("human", "{question}")
])
```

12. Output Parsers

Convert raw LLM output into structured format.

Common parsers:

Parser	Purpose
StrOutputParser	Text output
JSONOutputParser	JSON output
PydanticOutputParser	Structured schema
RegexParser	Pattern extraction

13. Structured Output with Pydantic

Example schema:

```
</> Python

from pydantic import BaseModel

class Answer(BaseModel):
    summary: str
    keywords: list[str]
```

Parser:

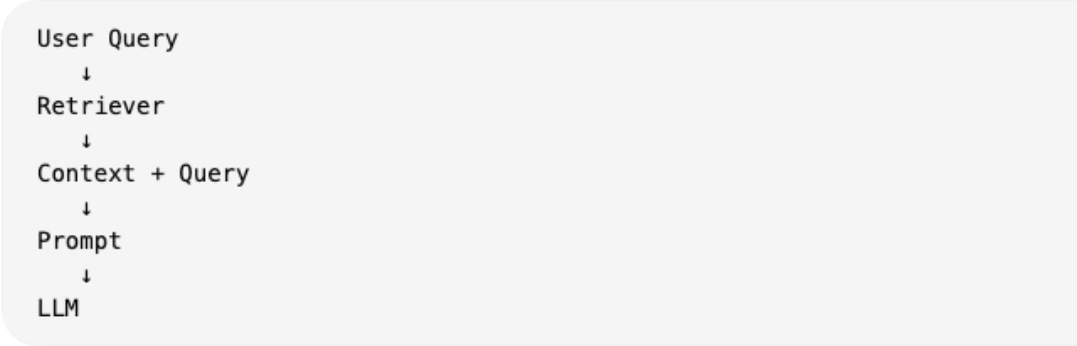
```
</> Python

parser = PydanticOutputParser(pydantic_object=Answer)
```

14. Retrieval-Augmented Generation (RAG) with LCEL

LCEL simplifies RAG pipelines.

Architecture:



Example:

```
</> Python

chain = (
    {"context": retriever, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

15. Streaming Responses

LCEL supports streaming.

Example:

```
</> Python

for chunk in chain.stream({"question": "Explain transformers"}):
    print(chunk)
```

Useful for:

- chat apps
- real-time UI

16. Async Execution

LCEL supports async workflows.

Example:

```
</> Python
result = await chain.ainvoke(input)
```

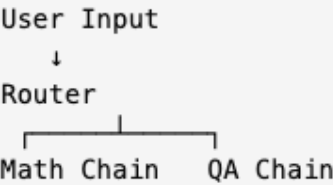
Async batch:

```
</> Python
results = await chain.abatch(inputs)
```

17. Branching Chains

Conditional execution using Router chains.

Example concept:



18. Error Handling

LCEL supports retry strategies.

Example:

```
</> Python
chain.with_retry()
```

Options include:

- exponential backoff
- max retries

19. Debugging and Observability

LCEL integrates with **LangSmith**.

Capabilities:

- trace chains
- visualize pipelines
- monitor latency
- debug prompts

Useful during production deployment.

20. LCEL Composition Patterns

Common patterns used in production systems.

Sequential Pipeline

Prompt → LLM → Parser

Parallel Pipeline

Input → Multiple Chains → Aggregated Output

RAG Pipeline

Query → Retriever → Prompt → LLM

Agent Tool Pipeline

User → Agent → Tool → LLM

21. Example: Full LCEL RAG Chain

```
</> Python
chain = (
    {
        "context": retriever,
        "question": RunnablePassthrough()
    }
    | prompt
    | llm
    | StrOutputParser()
)
```

22. LCEL vs Traditional LangChain Chains

Feature	Old Chains	LCEL
Composition	Imperative	Declarative
Streaming	Limited	Built-in
Async	Difficult	Native
Parallel execution	Hard	Simple
Debugging	Limited	LangSmith

23. Performance Benefits

LCEL improves:

- pipeline composability
- parallel execution
- async support
- debugging
- readability

Especially useful in:

- RAG systems
- agent frameworks
- LLM apps
- tool pipelines

24. Best Practices

1. Keep chains modular
2. Use structured output parsers
3. Implement retries for LLM calls
4. Use streaming for UI apps
5. Monitor pipelines using LangSmith

26. Key Libraries Used with LCEL

Common ecosystem tools:

- LangChain Core
- LangChain OpenAI
- LangGraph
- LangSmith
- FAISS / Chroma / Weaviate
- OpenAI / Anthropic / Mistral models

27. Where LCEL is Used

Typical applications:

- AI copilots
- Retrieval QA systems
- autonomous agents
- document analysis
- coding assistants
- customer support bots

25. Quick LCEL Mental Model

Input

↓

Prompt

↓

LLM

↓

Parser

↓

Output

Or for RAG:

Query

↓

Retriever

↓

Context + Query

↓

Prompt

↓

LLM

↓

Answer